

R Programming – Full Data Mining & Machine Learning Solutions

Complete practical-oriented notes with R code, calculations, expected interpretation and step-by-step procedure.

1. Apriori and FP-Growth – Support 3, Confidence 50%

Apriori finds frequent itemsets and then generates association rules. Here support 3 means an itemset must occur in at least 3 transactions. Confidence 50% means a rule must have confidence at least 0.50. FP-Growth is another frequent-itemset method that avoids generating all candidate itemsets explicitly.

Steps

Convert the transactions to an R transactions object.

Set support to $3/5 = 0.60$.

Run Apriori and inspect frequent itemsets/rules.

For FP-Growth, use an installed FP-Growth implementation and compare the resulting frequent itemsets.

R Code

```
# install.packages("arules")
library(arules)

transactions <- list(
  c("M", "O", "N", "K", "E", "Y"),
  c("D", "O", "N", "K", "E", "Y"),
  c("M", "A", "K", "E"),
  c("M", "U", "C", "K", "Y"),
  c("C", "O", "K", "I", "E")
)

tr <- as(transactions, "transactions")
inspect(tr)

# Apriori: support = 3/5 = 0.60, confidence = 0.50
rules_ap <- apriori(tr,
  parameter=list(supp=0.60, conf=0.50, minlen=2))
inspect(rules_ap)

# FP-Growth
# arules uses the Eclat/Apriori family for frequent itemsets;
# for an FP-Growth implementation, use the arulesSequences/
# appropriate FP-Growth package available in your R setup.
# The frequent itemsets can be compared with:
freq <- apriori(tr, parameter=list(supp=0.60,
                                   target="frequent itemsets"))
inspect(freq)
```

Result / Interpretation: With five transactions, support count 3 corresponds to support 60%. Rules are retained only when their confidence is at least 50%. The exact rule list depends on the supplied transaction table and implementation.

2. Maximum Number of Association Rules and Maximum Frequent-Itemset Size

For n distinct items, every non-empty proper subset can be an antecedent and the remaining non-empty subset can be a consequent. The maximum number of rules, allowing zero-support rules, is $3^n - 2^{(n+1)} + 1$. The largest possible frequent itemset is n items, provided that all n items occur together and minimum support is greater than zero.

Steps

Count the number of distinct items n .

Use $3^n - 2^{(n+1)} + 1$ for all possible non-empty antecedent/consequent rules.

A frequent itemset can contain at most all n distinct items.

R Code

```
# General R calculation
max_rules <- function(n) {
  3^n - 2^(n+1) + 1
}

n <- 5
max_rules(n)

# Maximum frequent-itemset size
maximum_itemset_size <- n
maximum_itemset_size
```

Result / Interpretation: For $n=5$, the maximum number of association rules is 180. The maximum possible frequent-itemset size is 5. If the missing 'above table' has a different number of distinct items, replace n accordingly.

3. Naive Bayes Classification – Training and Test Data

Naive Bayes predicts a categorical class using Bayes' theorem. Training data are used to estimate probabilities and test data are used to evaluate predictions.

Steps

Load a labelled dataset.

Split it into 70% training and 30% testing data.

Train Naive Bayes on the training data.

Predict the classes of the test data.

Compare predicted and actual classes and calculate accuracy.

R Code

```
# install.packages("e1071")
library(e1071)

data(iris)

set.seed(123)
idx <- sample(seq_len(nrow(iris)), size=0.7*nrow(iris))
train <- iris[idx, ]
test <- iris[-idx, ]

model_nb <- naiveBayes(Species ~ ., data=train)
pred <- predict(model_nb, test)

print(pred)
print(table(Actual=test$Species, Predicted=pred))

accuracy <- mean(pred == test$Species)
cat("Accuracy =", accuracy*100, "%\n")
```

Result / Interpretation: The confusion matrix shows correct and incorrect predictions. Accuracy is correct predictions divided by total test observations.

4. Decision Tree Classification – Training and Test Data

Decision trees recursively split the training data using attributes that best separate the classes. The final leaves contain predicted classes.

Steps

Split data into training and test sets.

Fit a classification tree using `rpart()`.

Display the tree.

Predict the test observations.

Calculate accuracy.

R Code

```
# install.packages("rpart")
# install.packages("rpart.plot")
library(rpart)
library(rpart.plot)

data(iris)

set.seed(123)
idx <- sample(seq_len(nrow(iris)), size=0.70*nrow(iris))
train <- iris[idx, ]
test <- iris[-idx, ]

tree_model <- rpart(Species ~ ., data=train, method="class")
rpart.plot(tree_model)

pred_tree <- predict(tree_model, test, type="class")
print(table(Actual=test$Species, Predicted=pred_tree))

accuracy_tree <- mean(pred_tree == test$Species)
cat("Decision Tree Accuracy =", accuracy_tree*100, "%\n")
```

Result / Interpretation: The tree diagram shows the splitting conditions. Accuracy depends on the random split and tree settings.

5. Diabetes.csv – Diabetes Trend by Age, Linear and Multiple Regression

This analysis studies whether blood pressure or diabetes-related outcome changes with age. Because the exact diabetes.csv was not supplied, the code assumes columns named Age, BloodPressure and Outcome. If your file uses different names, change the column names accordingly.

Steps

Read diabetes.csv.

Check the exact column names before analysis.

Plot Age against BloodPressure and add a regression line.

Fit simple linear regression.

Fit multiple regression using Age, BMI and Glucose.

Group ages and calculate the percentage with Outcome=1.

Plot the age-group diabetes rate.

R Code

```
diabetes <- read.csv("diabetes.csv")

# Inspect data
head(diabetes)
str(diabetes)
names(diabetes)

# Scatterplot: age vs blood pressure
plot(diabetes$Age, diabetes$BloodPressure,
     main="Blood Pressure vs Age",
     xlab="Age", ylab="Blood Pressure",
     pch=19)
abline(lm(BloodPressure ~ Age, data=diabetes))

# Simple linear regression
model1 <- lm(BloodPressure ~ Age, data=diabetes)
summary(model1)

# Multiple regression example
# Uses Age, BMI and Glucose to predict BloodPressure
model2 <- lm(BloodPressure ~ Age + BMI + Glucose,
             data=diabetes)
summary(model2)
```

```
# If Outcome is 0/1, calculate diabetes rate by age group
diabetes$AgeGroup <- cut(diabetes$Age,
                        breaks=c(0,20,30,40,50,60,100),
                        right=FALSE)

rate <- aggregate(Outcome ~ AgeGroup,
                  data=diabetes, FUN=mean)
rate$Percent <- rate$Outcome * 100
print(rate)

barplot(rate$Percent,
        names.arg=rate$AgeGroup,
        main="Diabetes Rate by Age Group",
        xlab="Age Group", ylab="Diabetes Rate (%)",
        las=2)
```

Result / Interpretation: The slope in the linear model tells how BloodPressure changes with Age. In multiple regression, each coefficient represents the relationship with the response while controlling for the other predictors. The exact numerical result requires the user's diabetes.csv.

6. Apriori on the Given Five Transactions – Support 50%, Confidence 80%

Transactions: T1=(M,O,N,K,E,Y), T2=(D,O,N,K,E,Y), T3=(M,A,K,E), T4=(M,U,C,K,Y), T5=(C,O,K,I,E). Minimum support is 2 of 5 transactions = 40% or higher; the question states 50% and then '(2)', which is internally inconsistent because 50% of 5 is 2.5. In practical Apriori, a count of 2 is commonly treated as the intended threshold. The code below uses support count 2/5 = 40% to match the stated '(2)'.

Steps

Enter the five transactions.

Set minimum support count to 2, i.e. $2/5 = 0.40$.

Generate all frequent itemsets.

Set minimum confidence to 0.80.

Inspect and sort the generated rules.

R Code

```
library(arules)

tr <- as(list(
  c("M", "O", "N", "K", "E", "Y"),
  c("D", "O", "N", "K", "E", "Y"),
  c("M", "A", "K", "E"),
  c("M", "U", "C", "K", "Y"),
  c("C", "O", "K", "I", "E")
), "transactions")

# Intended minimum support count = 2
freq <- apriori(tr,
  parameter=list(supp=2/5, target="frequent itemsets"))
inspect(freq)

rules <- apriori(tr,
  parameter=list(supp=2/5, conf=0.80, minlen=2))
inspect(sort(rules, by="confidence"))

# Show support and confidence
quality(rules)
```

Result / Interpretation: Important: if your teacher strictly interprets 50% literally, use $\text{supp}=0.50$. Since five transactions cannot have exactly 50% support, an itemset must occur in at least 3 transactions. If the worksheet explicitly says '(2)', use $2/5$ as above.

7. FP-Growth Tree for the Five Transactions

FP-Growth compresses transactions into a prefix tree. Items are usually ordered by descending frequency before insertion. The exact graphical tree depends on the chosen implementation.

Steps

Count the frequency of every item.

Remove items below minimum support count.

Order each transaction by global item frequency.

Insert the ordered transactions into an FP tree. Shared prefixes are represented by common nodes.

R Code

```
# Transaction frequencies
items <- list(
  c("M", "O", "N", "K", "E", "Y"),
  c("D", "O", "N", "K", "E", "Y"),
  c("M", "A", "K", "E"),
  c("M", "U", "C", "K", "Y"),
  c("C", "O", "K", "I", "E")
)

all_items <- unlist(items)
freq <- sort(table(all_items), decreasing=TRUE)
freq

# Keep items meeting the intended support count >= 2
freq_items <- names(freq[freq >= 2])
freq_items

# Sort each transaction by descending global frequency
ordered_transactions <- lapply(items, function(x)
  x[order(match(x, names(freq)), decreasing=TRUE)])
ordered_transactions
```

Result / Interpretation: For this dataset, K occurs in all five transactions; E, M, O and Y occur frequently enough to be important at a count-2 threshold. The exact tree drawing is produced by the FP-Growth package/tool used in the lab.

8. ARFF Dataset – Apriori and FP-Growth Rules

An ARFF file contains relation metadata, attribute definitions and data rows. In R, a transaction dataset can be represented directly and association rules can then be mined.

Steps

Create or import the ARFF data.

Convert transactions to an arules transaction object.

Set minimum support to 0.3333.

Set confidence to 0.60.

Rules with confidence ≥ 0.60 are accepted.

R Code

```
# Example transaction dataset
tr <- as(list(
  c("Bread", "Milk"),
  c("Bread", "Diaper", "Beer", "Eggs"),
  c("Milk", "Diaper", "Beer", "Cola"),
  c("Bread", "Milk", "Diaper", "Beer"),
  c("Bread", "Milk", "Diaper", "Cola"),
  c("Milk", "Diaper", "Beer"),
  c("Bread", "Milk", "Beer"),
  c("Bread", "Diaper", "Cola")
), "transactions")

# s = 33.33%, c = 60%
library(arules)

freq <- apriori(tr,
  parameter=list(supp=0.3333, target="frequent itemsets"))
inspect(freq)
rules <- apriori(tr,
```

```

parameter=list(supp=0.3333, conf=0.60, minlen=2))
inspect(sort(rules, by="confidence"))

# Accepted rules satisfy confidence >= 0.60
q <- quality(rules)
q$accepted <- q$confidence >= 0.60
q

```

Result / Interpretation: The example is a demonstration because the original ARFF dataset was not supplied. For your actual practical, replace the example transactions with your teacher's ARFF data.

9. Rule-Based Classification vs Decision Tree Classification

Both methods classify categorical outcomes. A rule-based classifier uses IF–THEN rules, while a decision tree uses hierarchical splits. Their accuracies can be compared on the same test set.

Steps

Use the same training and test observations for both algorithms.

Train the rule-based PART classifier.

Train the decision tree.

Predict the same test data.

Calculate both accuracies.

Plot the comparison.

R Code

```

# install.packages("RWeka")
# install.packages("rpart")
library(RWeka)
library(rpart)

data(iris)
set.seed(123)
idx <- sample(seq_len(nrow(iris)), size=0.70*nrow(iris))
train <- iris[idx, ]
test <- iris[-idx, ]

# Rule-based classifier: PART
part_model <- PART(Species ~ ., data=train)
part_pred <- predict(part_model, test)
part_acc <- mean(part_pred == test$Species)

# Decision tree
tree_model <- rpart(Species ~ ., data=train, method="class")
tree_pred <- predict(tree_model, test, type="class")
tree_acc <- mean(tree_pred == test$Species)

cat("Rule-based accuracy:", part_acc*100, "%\n")
cat("Decision tree accuracy:", tree_acc*100, "%\n")

acc <- c(part_acc, tree_acc)
barplot(acc*100,
        names.arg=c("Rule-Based", "Decision Tree"),
        main="Accuracy Comparison",
        ylab="Accuracy (%)")

```

Result / Interpretation: The higher bar indicates the algorithm with higher test accuracy for that particular split. Accuracy alone should not be the only criterion when comparing classifiers.

10. WEKA-style Preprocessing, Tree and Logistic Regression in R

WEKA practicals can be reproduced in R with equivalent classifiers. Preprocessing means loading data, checking missing values, converting categorical variables to factors and splitting data into training/testing sets.

Steps

Inspect structure and missing values.

Convert the target to a factor.

Split into training and testing data.

Train the decision tree.

Train multinomial logistic regression.

Compare test accuracy.

R Code

```
data(iris)

# Preprocessing
str(iris)
sum(is.na(iris))
iris$Species <- as.factor(iris$Species)

set.seed(123)
idx <- sample(seq_len(nrow(iris)), 0.70*nrow(iris))
train <- iris[idx, ]
test <- iris[-idx, ]

# Decision tree
library(rpart)
tree <- rpart(Species ~ ., data=train, method="class")
tree_pred <- predict(tree, test, type="class")
tree_acc <- mean(tree_pred == test$Species)

# Multinomial logistic regression
# install.packages("nnet")
library(nnet)
logit <- multinom(Species ~ ., data=train, trace=FALSE)
logit_pred <- predict(logit, test)
logit_acc <- mean(logit_pred == test$Species)

cat("Decision Tree Accuracy:", tree_acc*100, "%\n")
cat("Logistic Accuracy:", logit_acc*100, "%\n")
```

Result / Interpretation: R provides an equivalent implementation of the classification tasks commonly performed in WEKA. Exact accuracy depends on the dataset and random split.

Important Exam Notes

- **Support:** $\text{support}(X) = \text{number of transactions containing } X / \text{total number of transactions}$.
- **Confidence:** $\text{confidence}(X \rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$.
- **Apriori principle:** if an itemset is frequent, all of its non-empty subsets must also be frequent.
- **FP-Growth:** builds an FP-tree and mines frequent patterns without explicit candidate generation in the same way as Apriori.
- **Naive Bayes:** uses Bayes' theorem and assumes conditional independence of predictors given the class.
- **Decision Tree:** recursively splits data to reduce impurity, commonly using Gini impurity or information gain.
- **Linear regression:** predicts a numerical response using one predictor.
- **Multiple regression:** predicts a numerical response using two or more predictors.
- **Accuracy:** $(\text{number of correct predictions} / \text{total predictions}) \times 100$.
- **Important:** Exact numerical outputs for diabetes.csv and the missing first transaction table cannot be stated without those actual files/data.